DCS 2026 Spring
Final Project – Appendix

TA : 陳孟頡　Advisor : Tian-Sheuan Chang

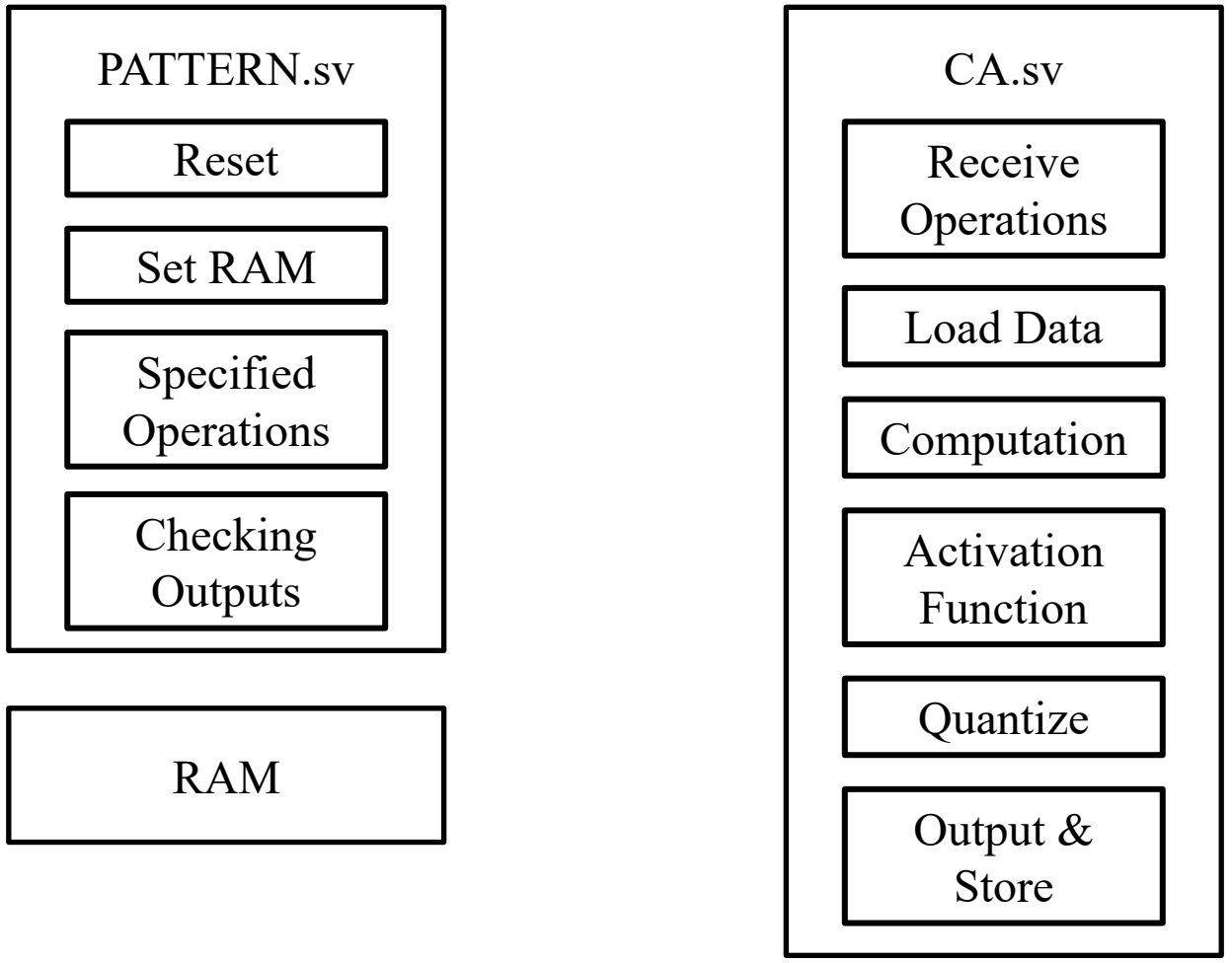
Date : 2026.05.18

Outline

- Computation Flow
- Operation Types
- RAM

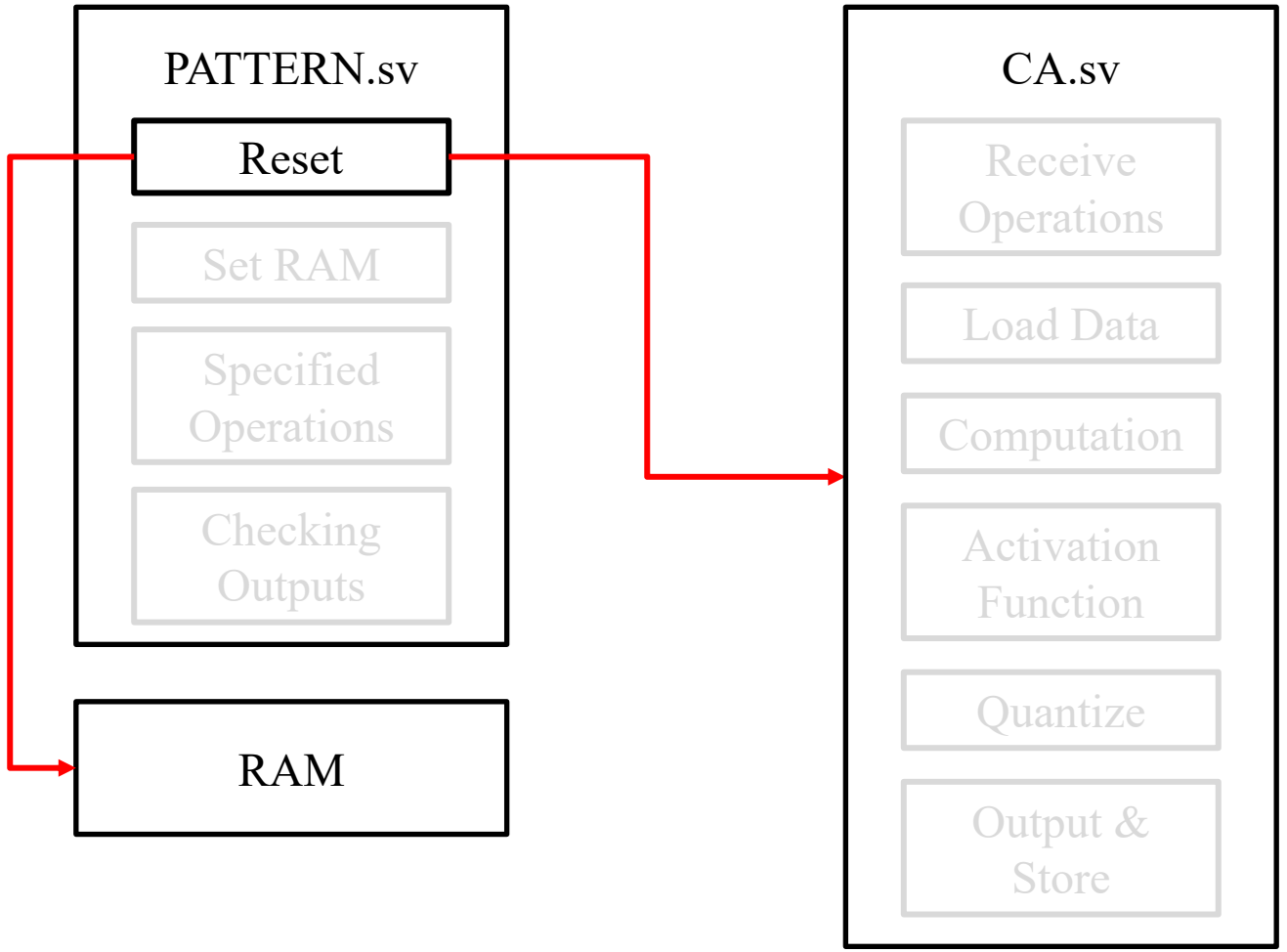
Computation Data Flow

- 此圖為 PATTERN 以及 CA 所(應)具備的功能



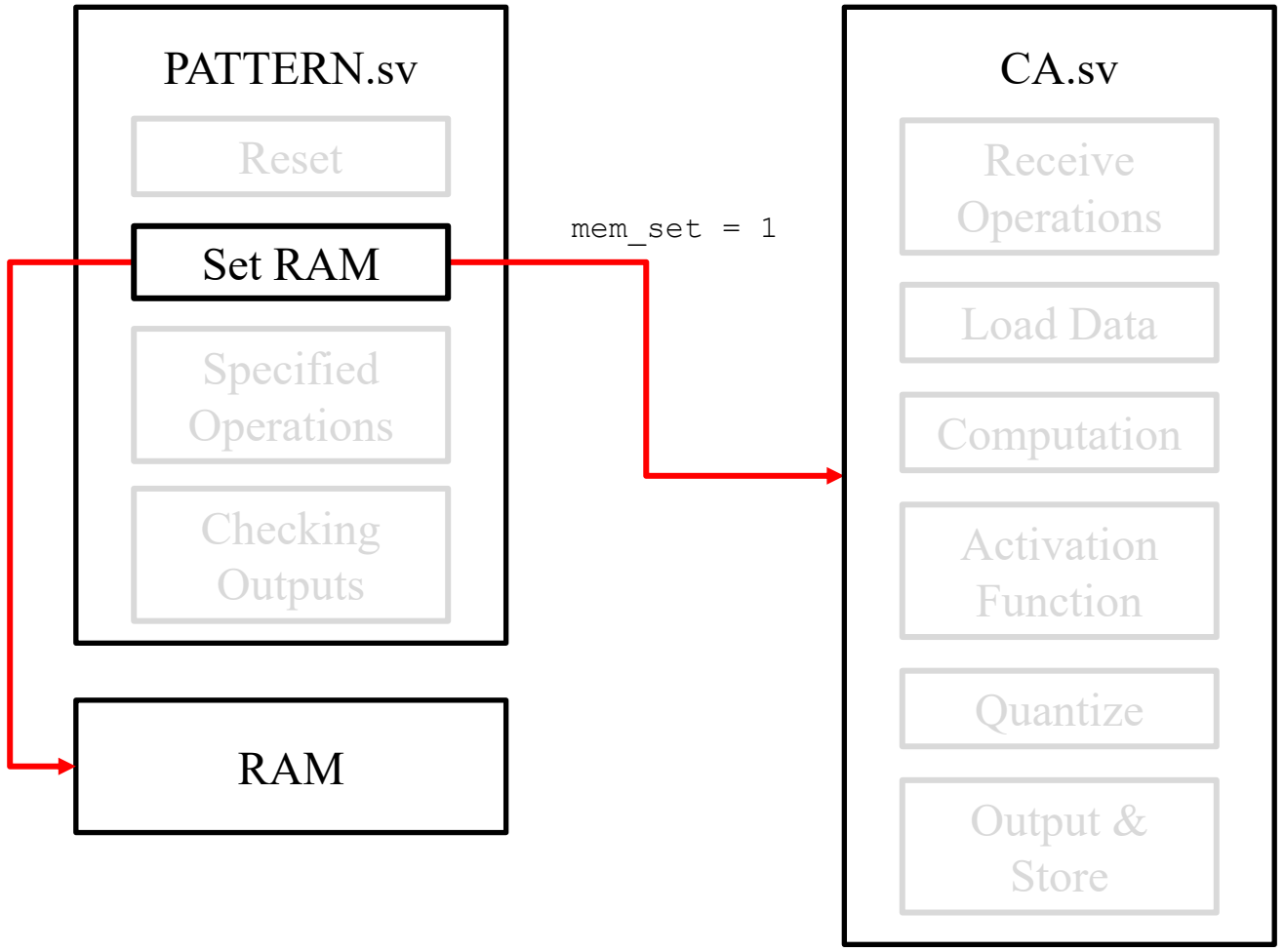
Computation Data Flow

- Step 1: PATTERN 以 `rst_n` 訊號 reset RAM 和 CA



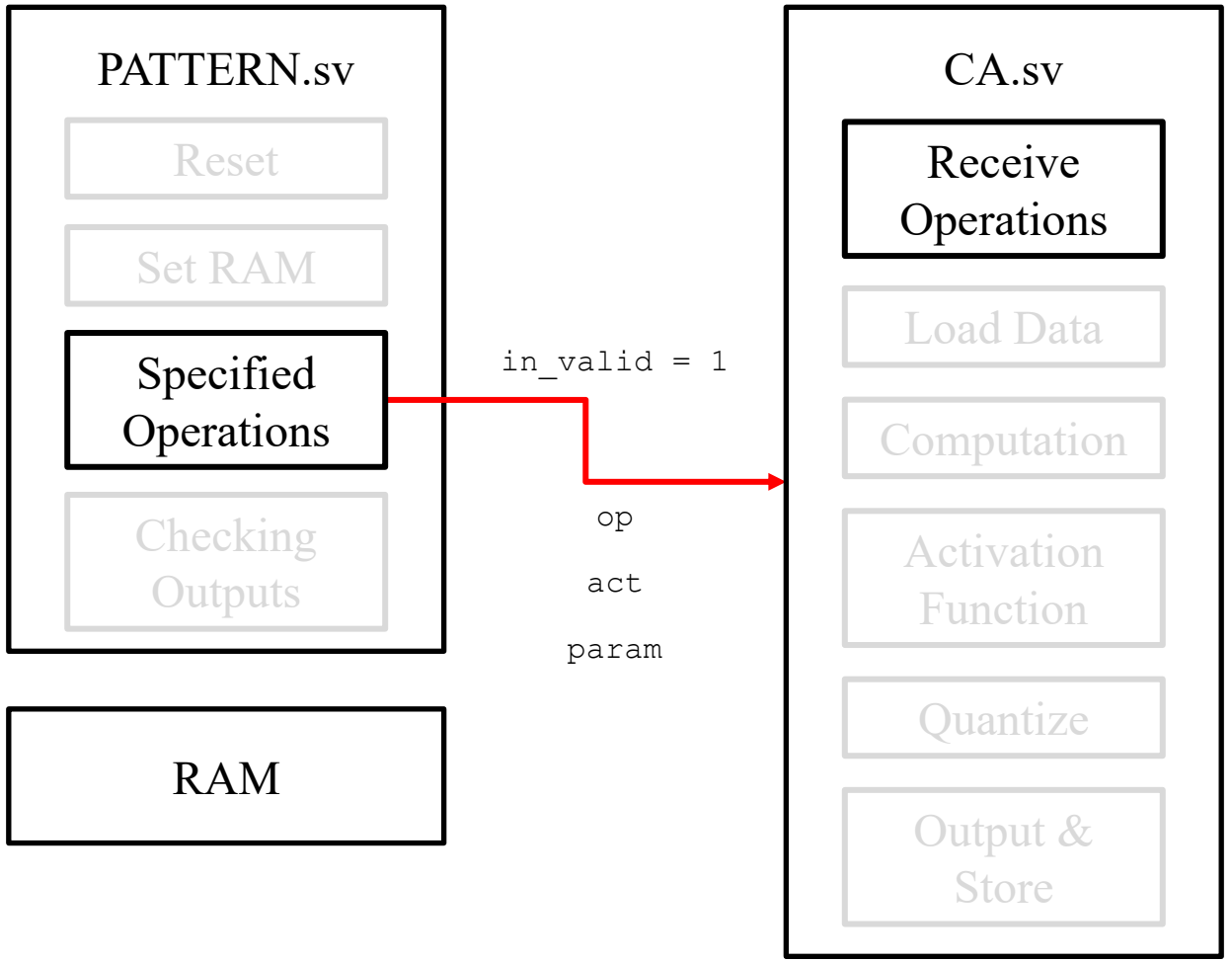
Computation Data Flow

- Step 2: PATTERN 設定此輪 RAM 內資料並拉起 `mem_set` 通知 CA



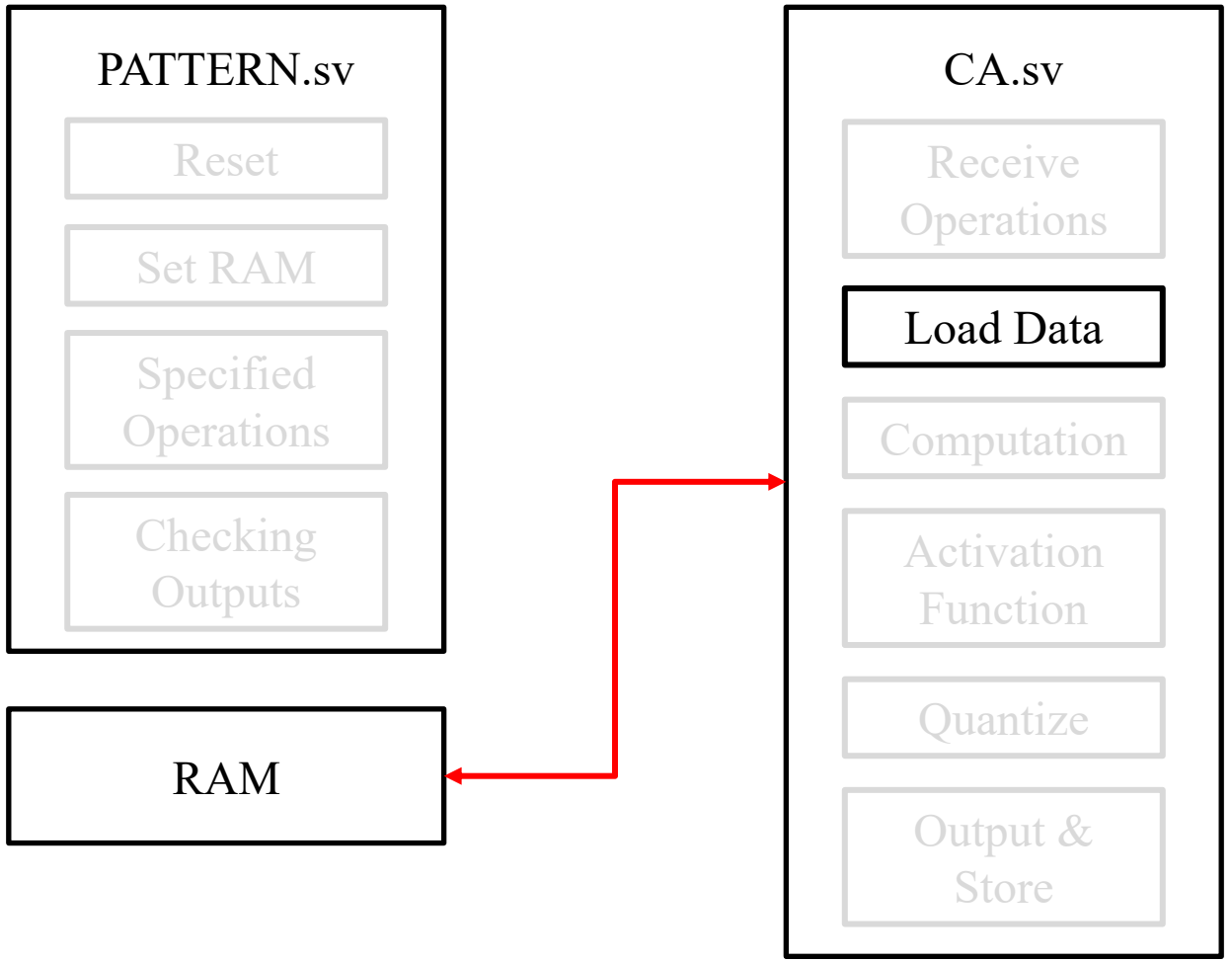
Computation Data Flow

- Step 3: PATTERN 將要使用的 operation set 送到 CA



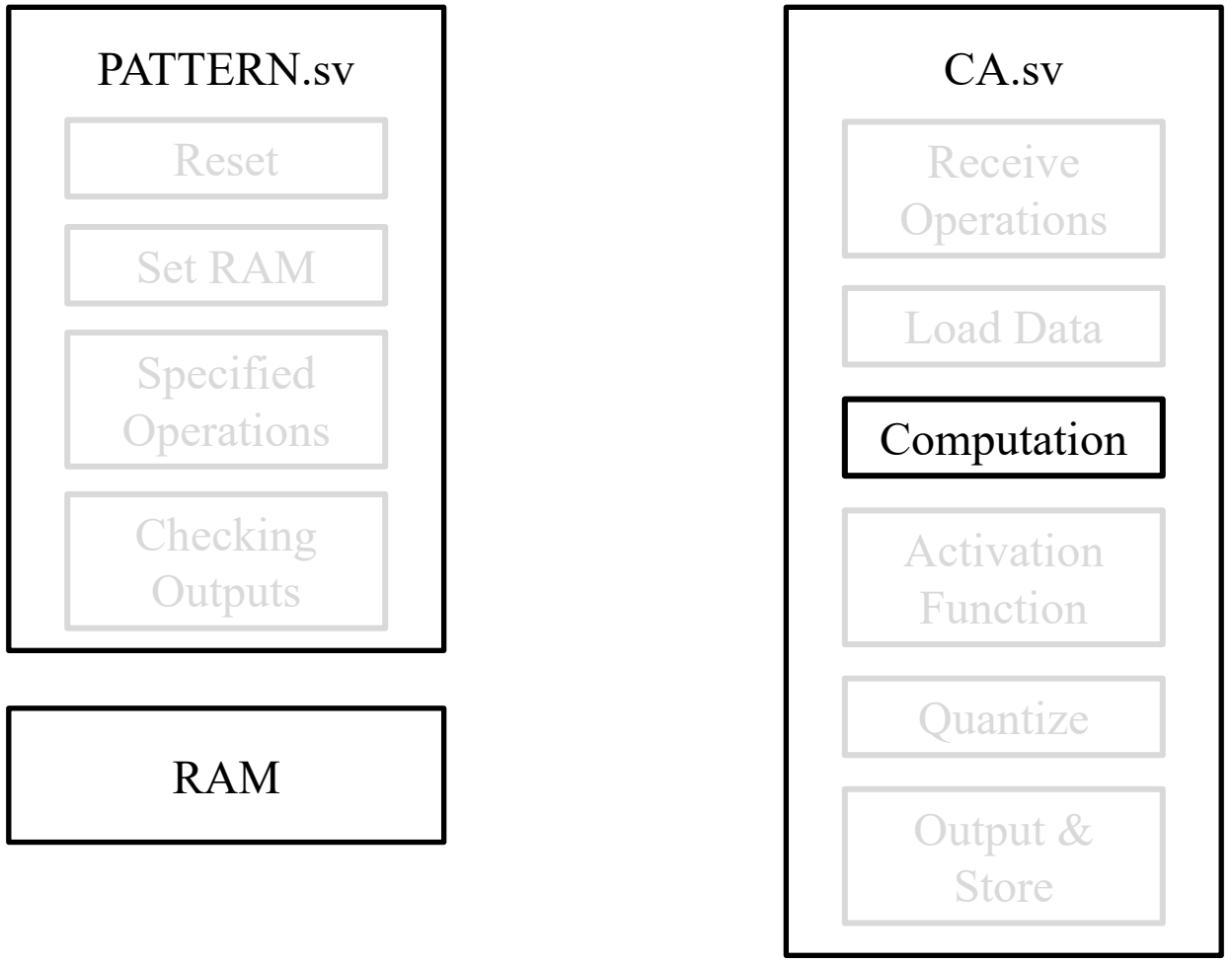
Computation Data Flow

- Step 4: 收到要執行的 operation set 後，CA 從 RAM 讀取 input data



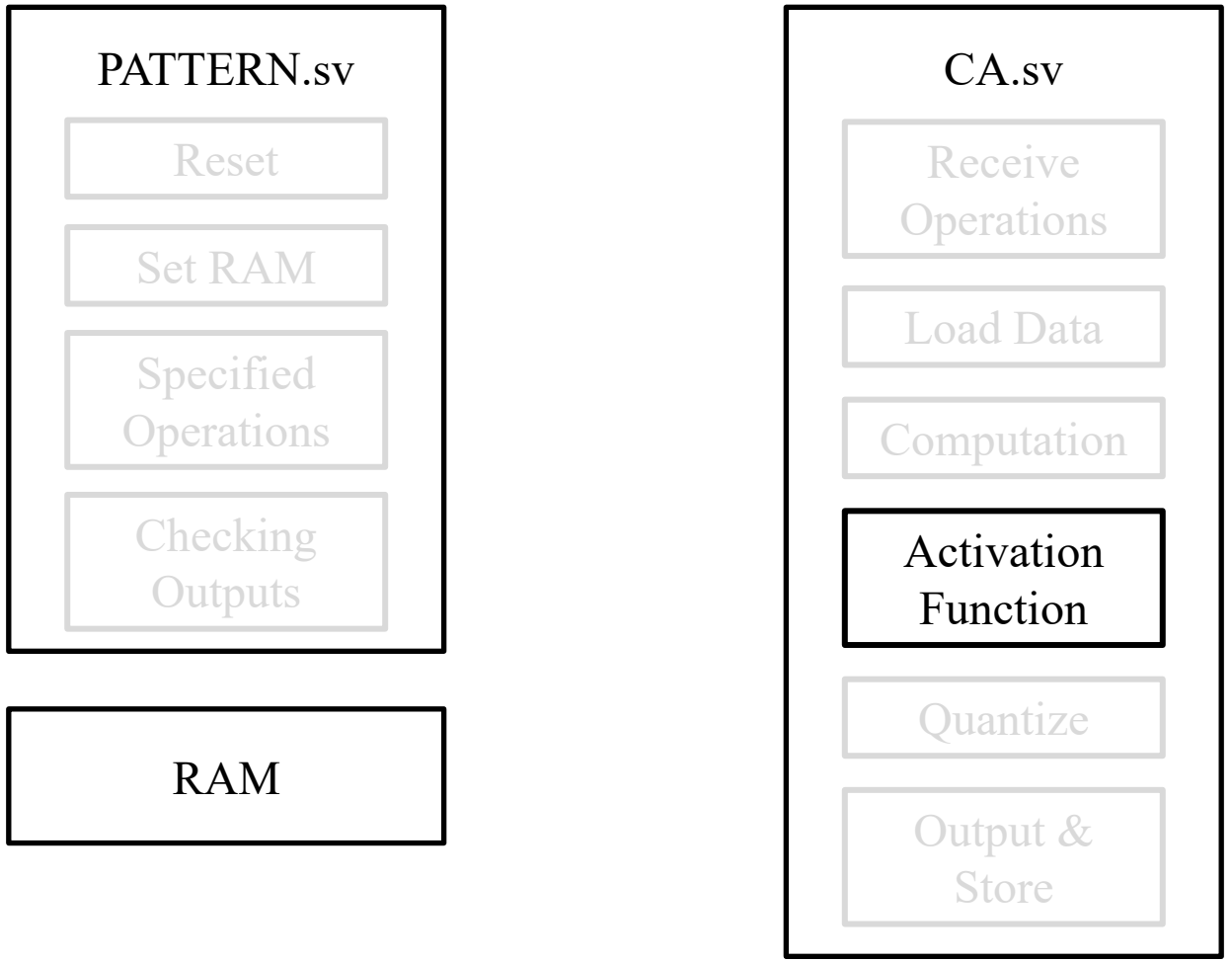
Computation Data Flow

- Step 5-1: CA 執行指定運算



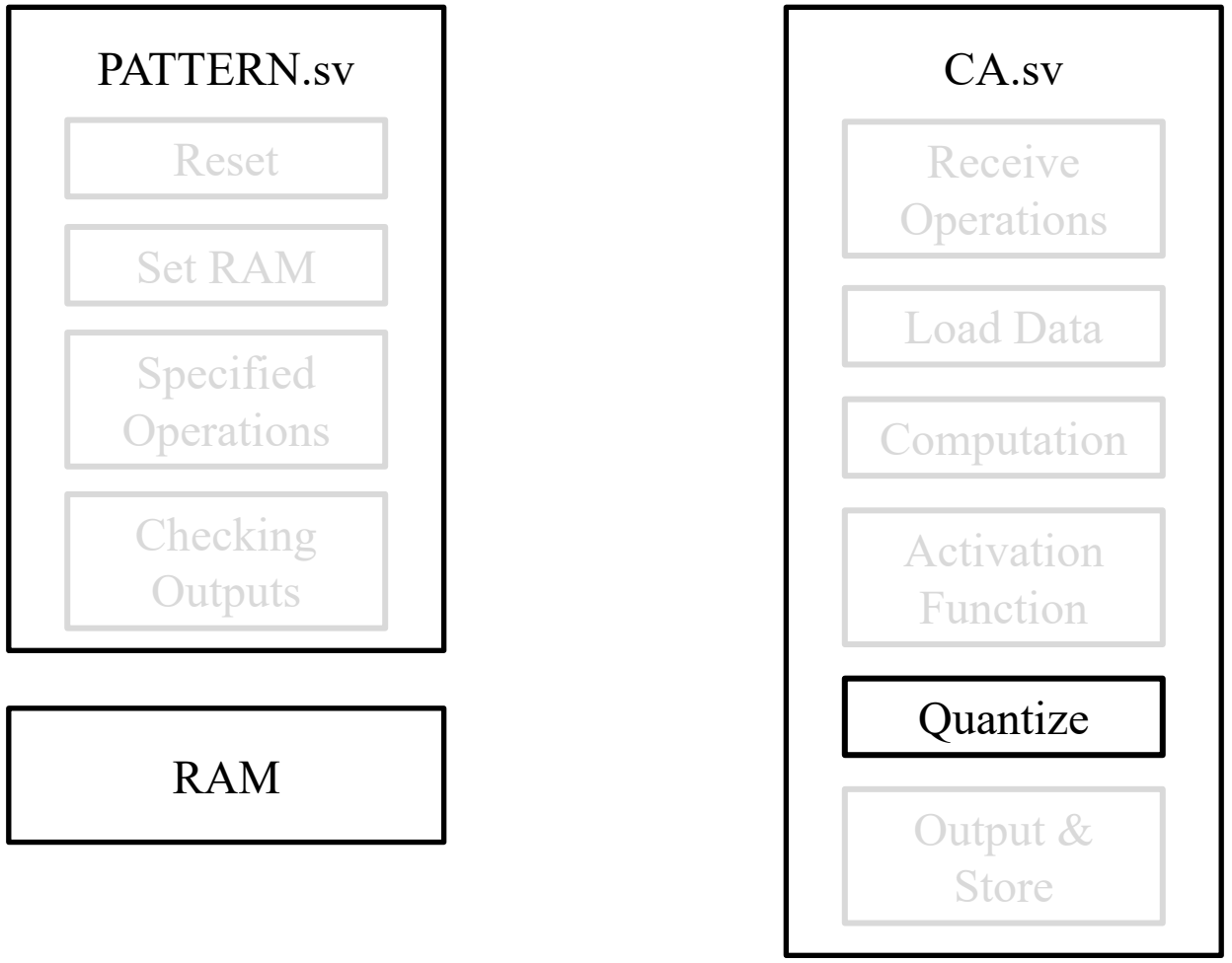
Computation Data Flow

- Step 5-2: CA 執行指定 activation function



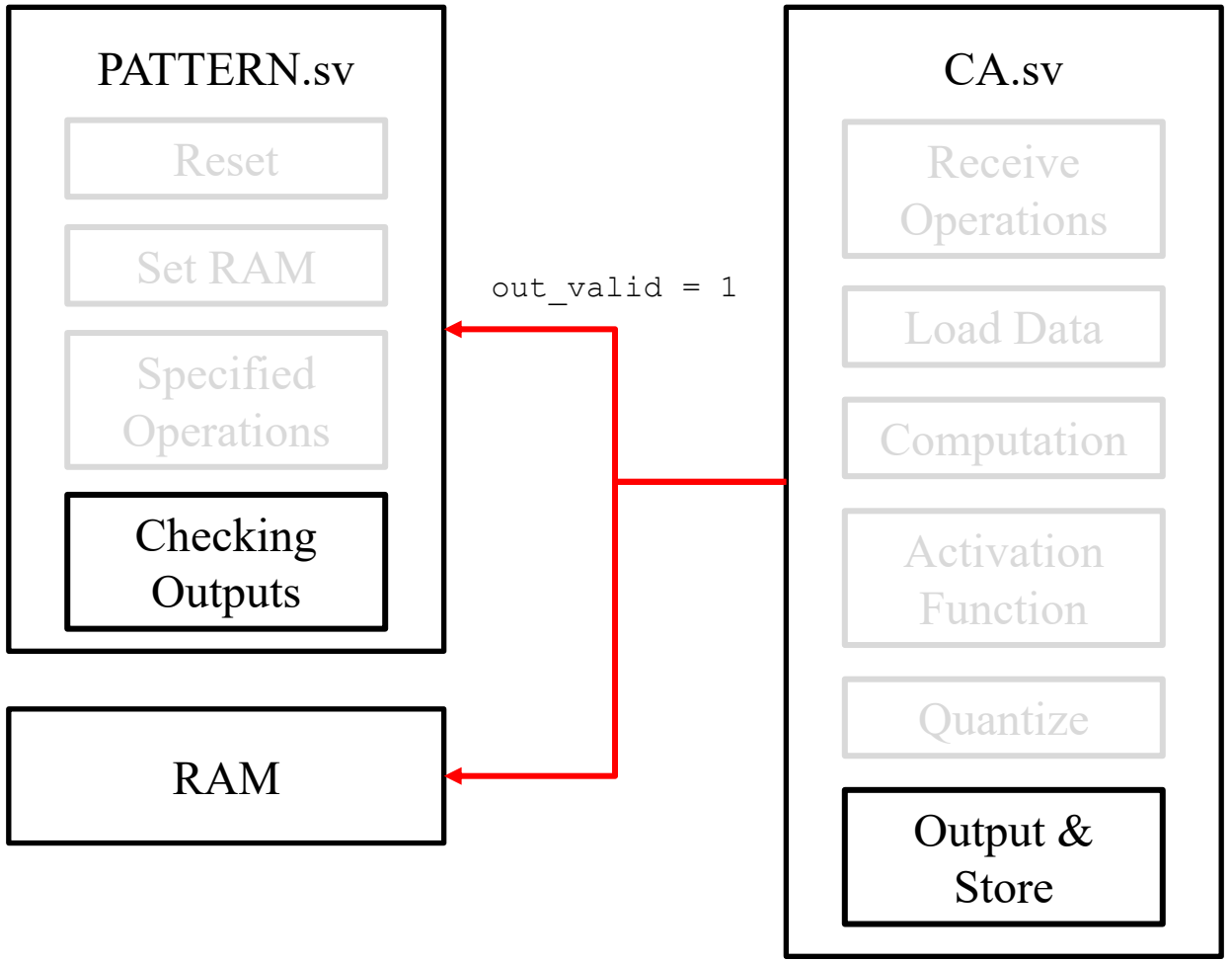
Computation Data Flow

- Step 5-3: CA 將經過 activation function 的 matrix 重新 quantize 成 4-bit



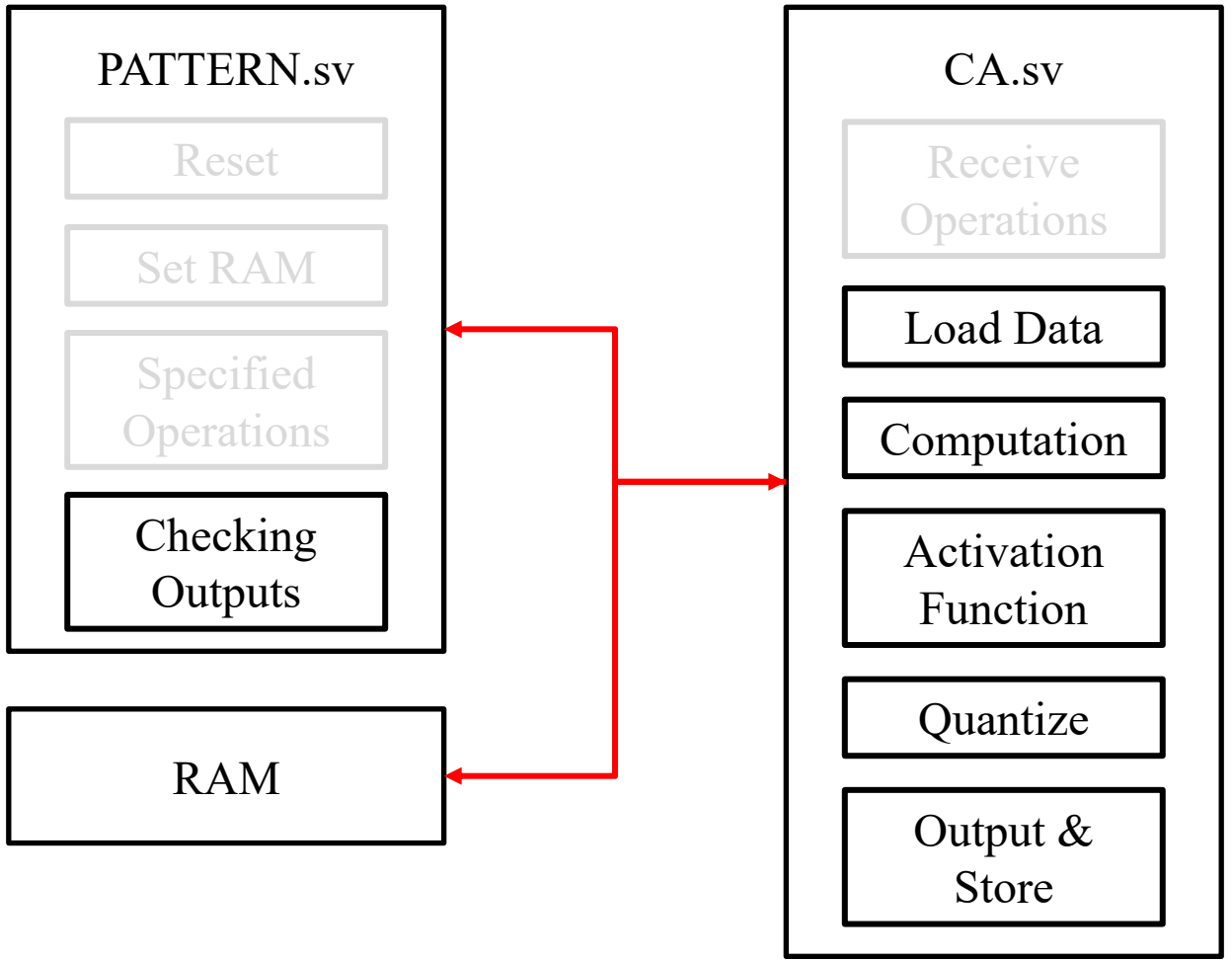
Computation Data Flow

- Step 6: CA 將運算完的 matrix 存回 RAM 原本位址，並輸出 **last token** 給 PATTERN



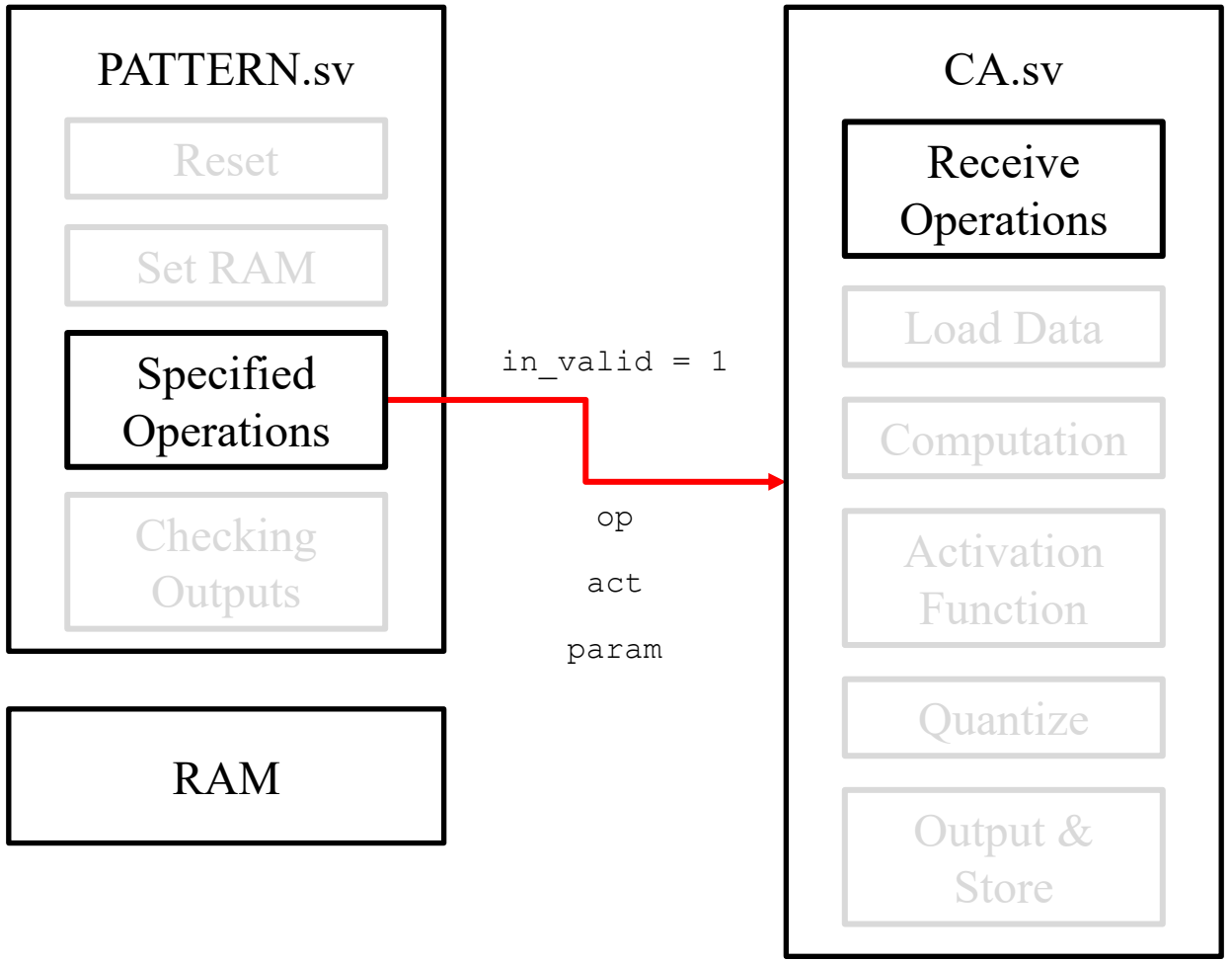
Computation Data Flow

- 對於每組 operation set，要將 RAM 中所有 input matrix 都運算完畢 (共有 256 筆)



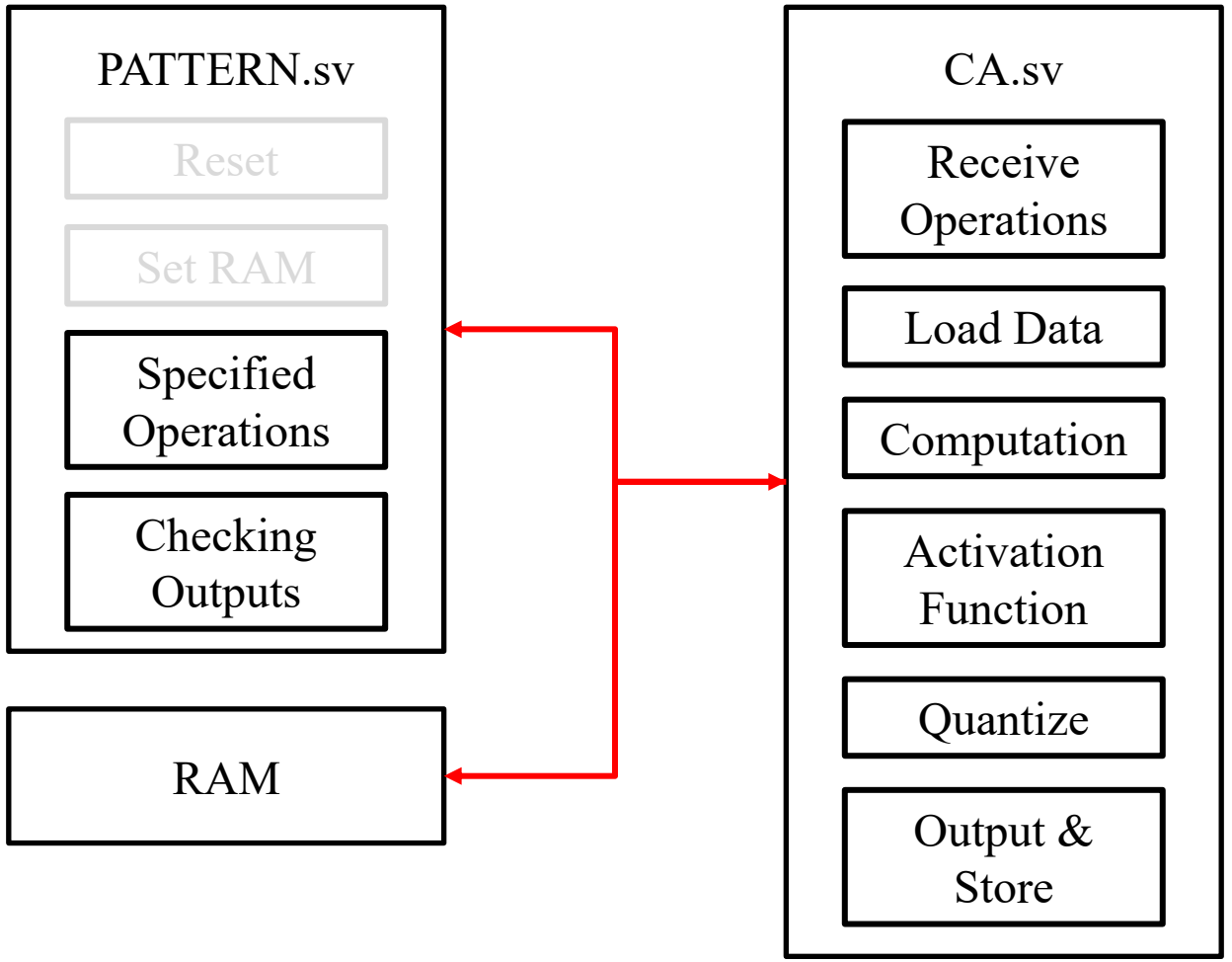
Computation Data Flow

- Step 7: 當此 operation set 運算完畢，PATTERN 會送一組新的 set 給 CA



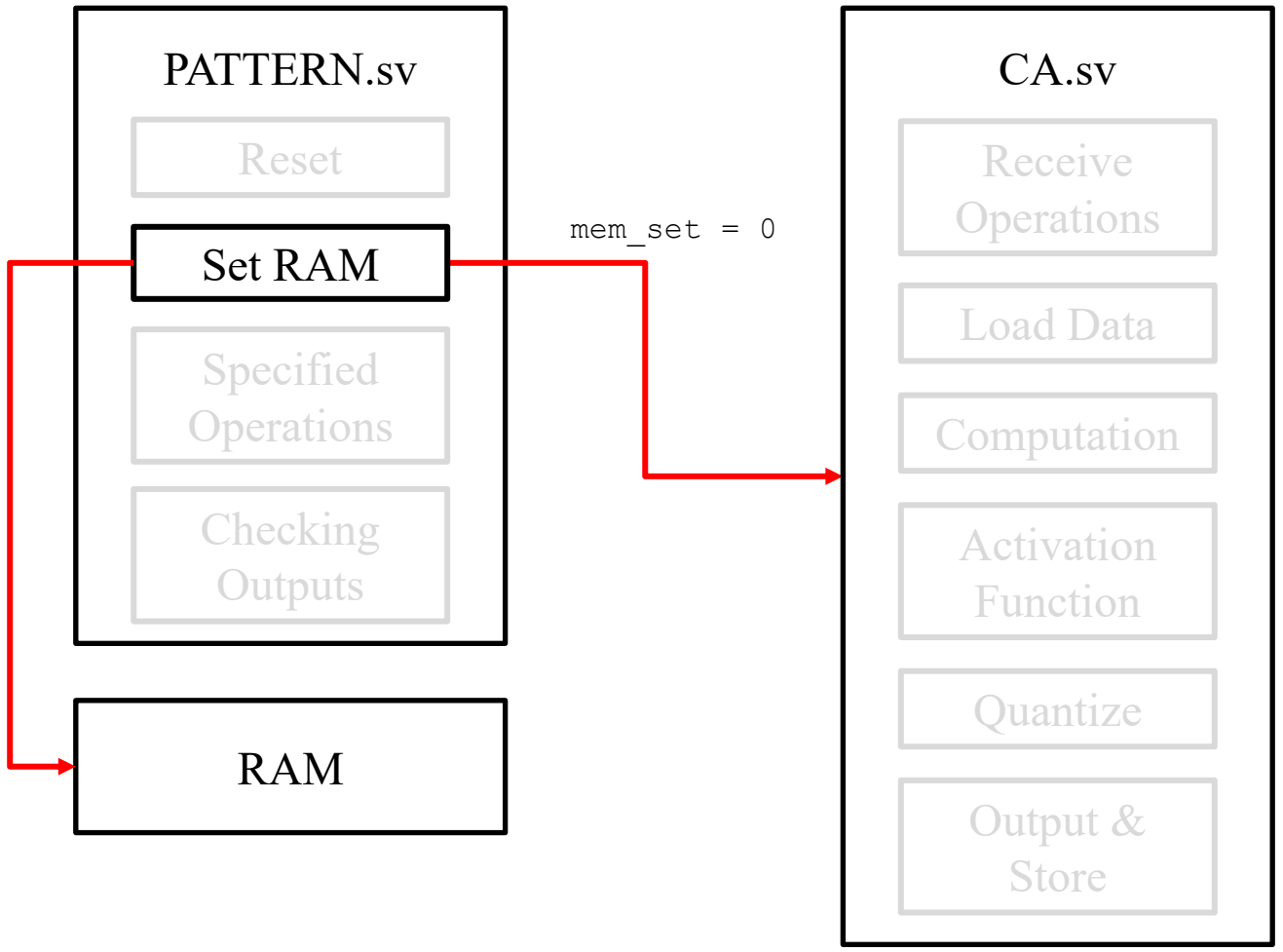
Computation Data Flow

- 對於每輪 RAM data，PATTERN 會測試多組不同的 operation set



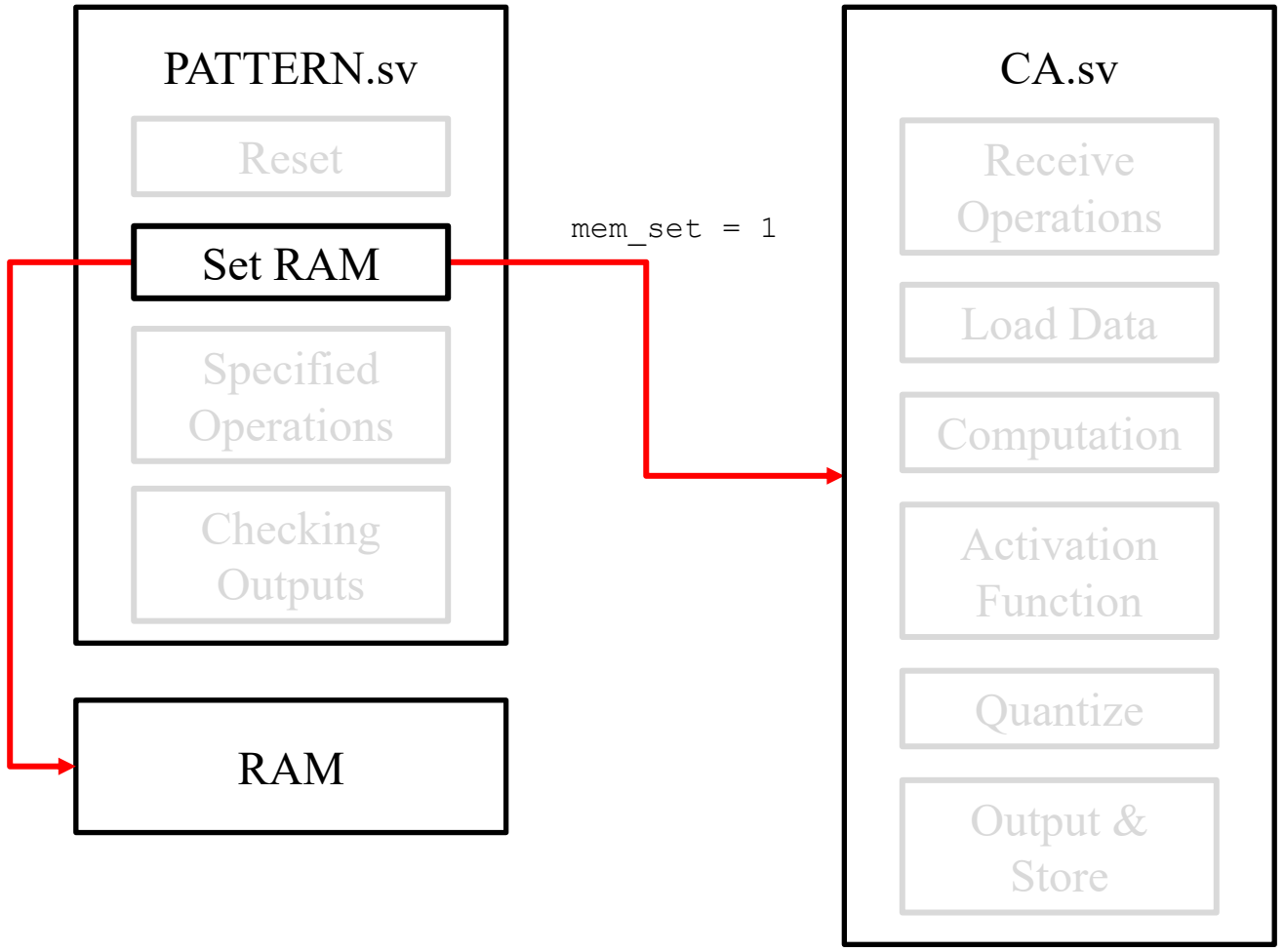
Computation Data Flow

- Step 8: 當此輪 RAM data 所有 operation set 完成, PATTERN 會拉低 `mem_set`



Computation Data Flow

- 回到 Step 2 (設定新一輪 RAM data)，並以此循環

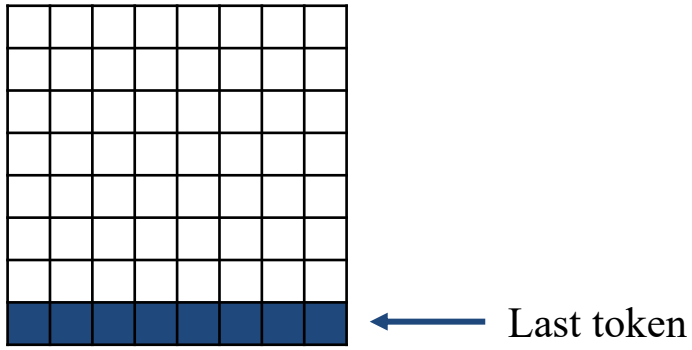


Summary

- 對於整個 PATTERN
  - Demo 會測試 N 輪 RAM data
  - 當 mem_set 抬起代表還有下一輪
- 對於每輪 RAM data
  - 需運算 M 組 operation set
  - 當 mem_set=1 代表此輪 RAM data 仍有 operation set 要計算

Summary

- 對於每組 operation set
  - 需將 RAM 中 256 筆 input matrix 都算過一次
  - 每筆 input matrix 經過 computation -> activation function -> quantize 後需存回 RAM 中來作為下一組 operation set 的 input data
- Demo 會有 Hidden Cases
- Last token 指的是 last row of the matrix



Outline

- Computation Flow
- Operations Types
- RAM

Operation Types

- 1 組 Operation set = 1 個 computation + 1 個 activation function

[1:0] op	Computation Types
2’b00	FFN
2’b01	Convolution
2’b10	Single-Head Attention
2’b11	Multi-Head Attention

[1:0] act	Activation Functions
2’b00	ReLU
2’b01	RAT
2’b10	CAT
2’b11	BAT

Computation Pseudo Codes

- FFN (linear projection):

```
N = 8

for i in range(N):
    for j in range(N):
        accumulator = 0

        for k in range(N):
            accumulator = accumulator + (input[i][k] * weight[k][j])

        output[i][j] = accumulator
```

Computation Pseudo Codes

- Convolution (kernel size=3*3, zero padding=1, stride=1):

```
N = 8
K = 3
padding = 1
stride = 1
N_padded = N + (2 * padding)

# zero padding
for i in range(N_padded):
    for j in range(N_padded):
        padded_input[i][j] = 0

for i in range(N):
    for j in range(N):
        padded_input[i + padding][j + padding] = input[i][j]

# convolution
for i in range(N):
    for j in range(N):
        accumulator = 0

        for ki in range(K):
            for kj in range(K):
                in_i = (i * stride) + ki
                in_j = (j * stride) + kj
                accumulator = accumulator + (padded_input[in_i][in_j] * kernel[ki][kj])

        output[i][j] = accumulator
```

Computation Pseudo Codes

- Single-Head Attention:

```
# Step 1: Input -> Q, K, V (Linear Projection)
for i in range(N):
    for j in range(D):
        for k in range(D):
            Q[i][j] = Q[i][j] + (input[i][k] * Weight_q[k][j])
            K[i][j] = K[i][j] + (input[i][k] * Weight_k[k][j])
            V[i][j] = V[i][j] + (input[i][k] * Weight_v[k][j])

# Step 2: Quantize to 4-bit
Q_q[i][j] = quantize_to_int4(Q[i][j])
K_q[i][j] = quantize_to_int4(K[i][j])
V_q[i][j] = quantize_to_int4(V[i][j])

# Step 3: Attention Score = Q * K^T
for i in range(N):
    for j in range(N):
        accumulator = 0

        for k in range(D):
            accumulator = accumulator + (Q_q[i][k] * K_q[j][k])

        score[i][j] = accumulator

# Step 4: Partial = act(score)
partial = activation_function(score)

# Step 5: Attention context = Partial * V
for i in range(N):
    for j in range(D):
        accumulator = 0

        for k in range(N):
            accumulator = accumulator + (partial[i][k] * V_q[k][j])

        attn_ctx[i][j] = accumulator
```

```
def activation_function(score_1):
    N = 8
    partial_1 = [[0 for _ in range(N)] for _ in range(N)]

    for i in range(N):
        for j in range(N):
            if score_1[i][j] >= 0:
                partial_1[i][j] = score_1[i][j]
            else:
                partial_1[i][j] = score_1[i][j] >> 2

    return partial_1
```

Computation Pseudo Codes

- Multi-Head Attention:

```
head_dim = 4

# Step 1 & 2 is the same with single-head attention
# Step 3: Attention Score = Q * K^T
for i in range(N):      # head 1
    for j in range(N):
        accumulator = 0
        for k in range(0, head_dim):
            accumulator = accumulator + (Q_q[i][k] * K_q[j][k])

        score_1[i][j] = accumulator

for i in range(N):      # head 2
    for j in range(N):
        accumulator = 0
        for k in range(head_dim, D):
            accumulator = accumulator + (Q_q[i][k] * K_q[j][k])

        score_2[i][j] = accumulator

# Step 4: Partial = act(score)
partial_1 = activation_function(score_1)
partial_2 = activation_function(score_2)

# Step 5: Attention context = Partial * V
for i in range(N):      # head 1
    for j in range(0, head_dim):
        accumulator = 0
        for k in range(N):
            accumulator = accumulator + (partial_1[i][k] * V_q[k][j])

        attn_ctx[i][j] = accumulator

for i in range(N):      # head 2
    for j in range(head_dim, D):
        accumulator = 0
        for k in range(N):
            accumulator = accumulator + (partial_2[i][k] * V_q[k][j])

        attn_ctx[i][j] = accumulator
```

Computation Parameters

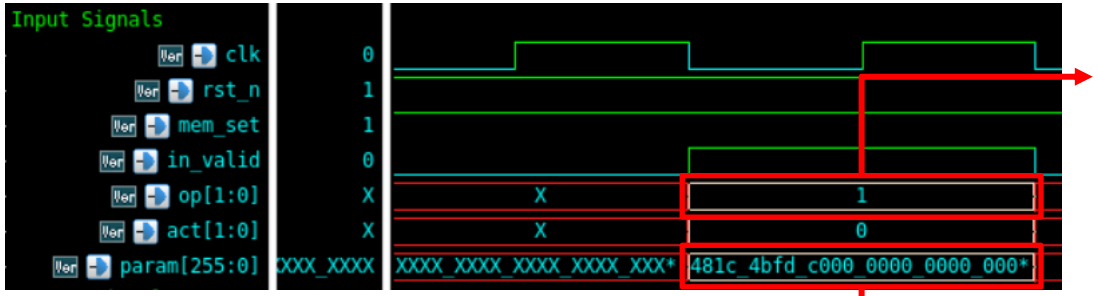
- 所有 parameter 皆為 signed 4-bit.
- FFN 有 8*8 = 64 parameters
- Convolution 有 3*3 = 9 parameters
- Attention 有 8*8*3(Q/K/V) = 192 parameters

Computation Parameters

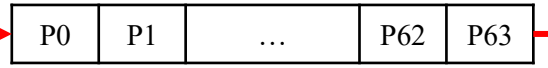
- For FFN and Convolution
  - `in_valid` high for 1 cycle.
  - Weight/Kernel given in raster scan order (from MSB to LSB).
- For Attention:
  - `in_valid` high for 3 cycle.
  - Q/K/V weight parameters given in raster scan order (1st cycle: Q, 2nd cycle: K, 3rd cycle: V).

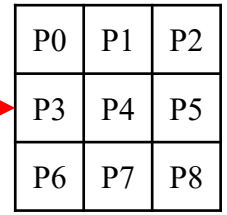
Computation Parameters

- FFN and Convolution

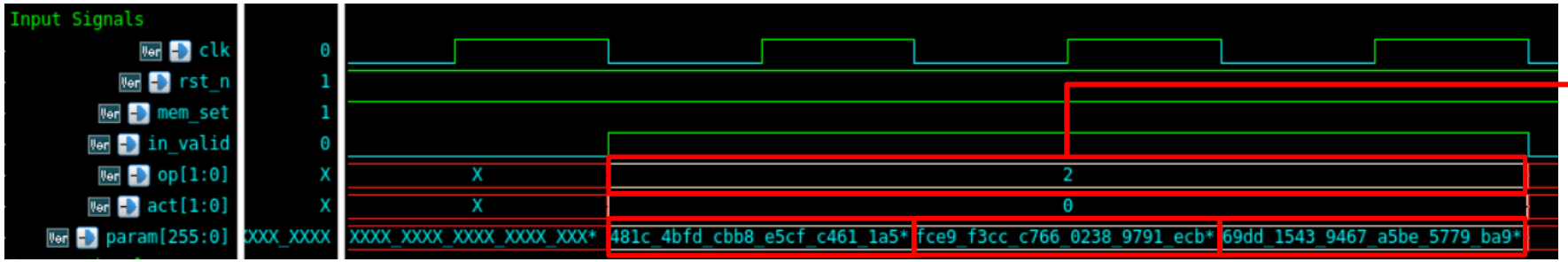


Convolution

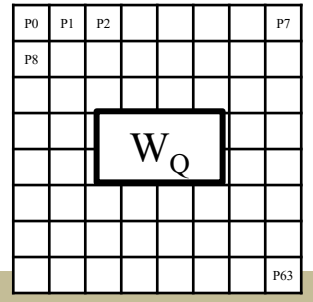


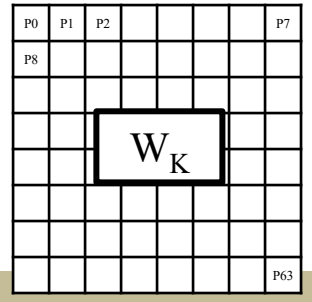


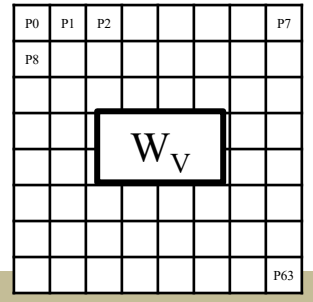
- Attention:



Single-Head Attention







Activation Functions

- ReLU:
  - If input value >=0, output that value, otherwise, output 0.
- RAT (Row-wise Average Threshold):
  - Computes the average of the row, and use it as the threshold. Values greater than or equal to this threshold are kept; others are divide by 8.
- CAT (Column-wise Average Threshold):
  - Similar to RAT, but operates on each column of a matrix.
- BAT (Block Average Threshold):
  - Similar to RAT, but operates on each block of a matrix (block size = 4).

Activation Function Pseudo Codes

```python
def ReLU(i_mtx: np.ndarray) -> np.ndarray:
    """
    Parameters
    ----------
    i_mtx : np.ndarray        # shape (L, D)

    Returns
    -------
    np.ndarray                # shape (L, D), same dtype/device
    """
    return np.maximum(0, i_mtx)

def RAT(i_mtx: np.ndarray) -> np.ndarray:
    """
    Parameters
    ----------
    i_mtx : np.ndarray        # shape (L, D)

    Returns
    -------
    np.ndarray                # shape (L, D), same dtype/device
    """
    row_means = i_mtx.mean(axis=1, keepdims=True)         # (L, 1)
    return np.where(i_mtx < row_means, i_mtx/8, i_mtx)
```

```python
def CAT(i_mtx: np.ndarray) -> np.ndarray:
    """
    Parameters
    ----------
    i_mtx : np.ndarray        # shape (L, D)

    Returns
    -------
    np.ndarray                # shape (L, D), same dtype/device
    """
    col_means = i_mtx.mean(axis=0, keepdims=True)         # (1, D)
    return np.where(i_mtx < col_means, i_mtx/8, i_mtx)

def BAT(i_mtx: np.ndarray, block_size: int = 4) -> np.ndarray:
    """
    Parameters
    ----------
    i_mtx : np.ndarray        # shape (L, D)

    Returns
    -------
    np.ndarray                # shape (L, D), same dtype/device
    """
    L, D = i_mtx.shape

    # 透過 reshape 將矩陣切割成多個 block_size * block_size 的區塊
    reshaped_mtx = i_mtx.reshape(L // block_size, block_size, D // block_size, block_size)

    # 計算每個 block 的平均值 (沿著 block 內部的維度 axis=1 與 axis=3)
    block_means = reshaped_mtx.mean(axis=(1, 3), keepdims=True)

    thresholded_mtx = np.where(reshaped_mtx < block_means, reshaped_mtx/8, reshaped_mtx)
    return thresholded_mtx.reshape(L, D)
```

Quantization

- 本次 final project 中所有 quantization 使用 PoT Quantization:
  - 此方法為動態且硬體友善的張量量化機制，透過尋找局部最大值，來得到最接近的 2 冪次方作為縮放比例，演算法如右圖
  - 用於 attention 運算及 activation function 之後
  - Quantization 輸入為任意 bit width
  - Quantization **輸出固定為 signed 4-bit**

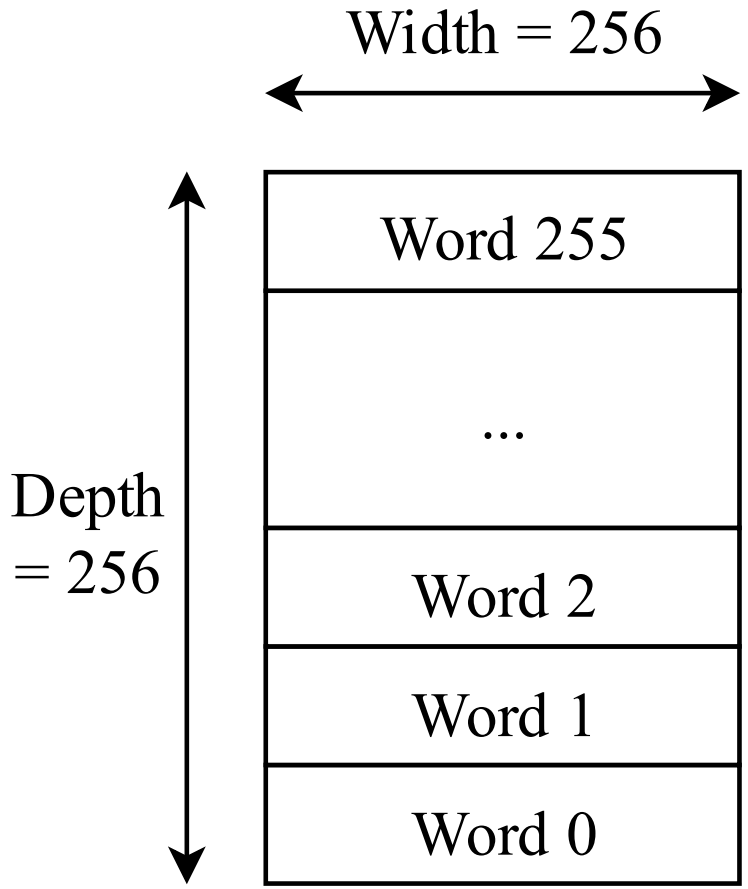
```python
DATA_WIDTH  = 8    # signed input bit-width
OUT_MAX     =  7   # 2^(4-1) - 1
OUT_MIN     = -8   # -2^(4-1)
LOG2_OUT_MAX = 2   # OUT_WIDTH - 2 = 2，即輸出最大值 7 ≈ 2^2

def quantize_pot(arr: list[list[int]]) -> list[list[int]]:

    # ── Step 1: Find maximum absolute value ──
    max_abs = 0
    for row in arr:
        for x in row:
            abs_val = abs(x)
            if abs_val > max_abs:
                max_abs = abs_val

    # ── Step 2: Compute PoT shift via MSB position ─
    shift = 0
    if max_abs > 0:
        i = max_abs.bit_length() - 1   # 等效 CLZ，找 MSB 位置
        shift = max(i - LOG2_OUT_MAX, 0)

    # ── Step 3: Arithmetic right-shift + clamp ──
    result = [[0] * 8 for _ in range(8)]
    for r, row in enumerate(arr):
        for c, x in enumerate(row):
            scaled = x >> shift
            result[r][c] = max(OUT_MIN, min(OUT_MAX, scaled))

    return result
```

Outline

- Computation Flow
- Computation and Activation Functions
- RAM

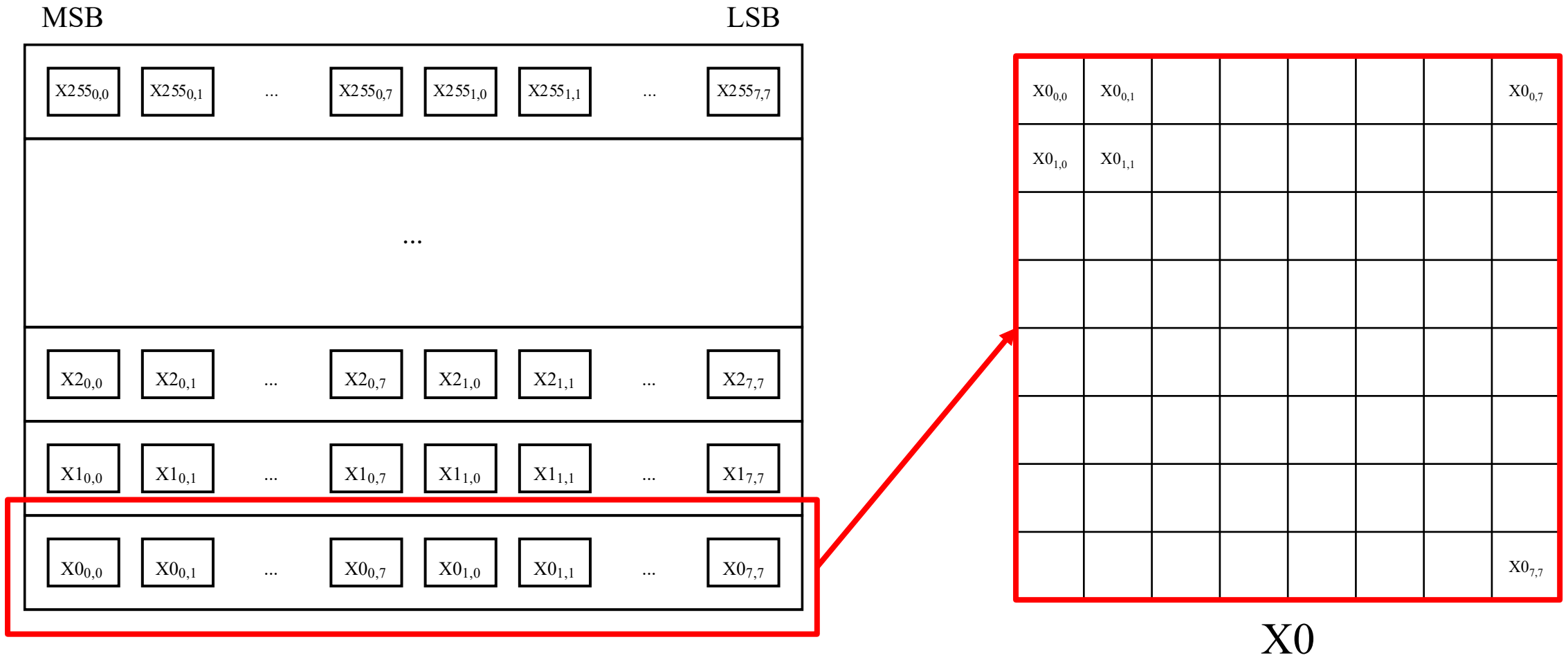
Pseudo RAM : System Specifications

- Dual-port Memory Architecture:
  - Features independent ports to support simultaneous Read and Write operations.
- Memory Configuration:
  - **Depth**: 256 words
  - **Data**: Width: 256 bits
- **Note**: Do not modify RAM or you might fail at demo.



Pseudo RAM : System Specifications

- Each word of RAM is a 8*8 data matrix (each data is signed 4-bit).



Pseudo RAM : System Specifications

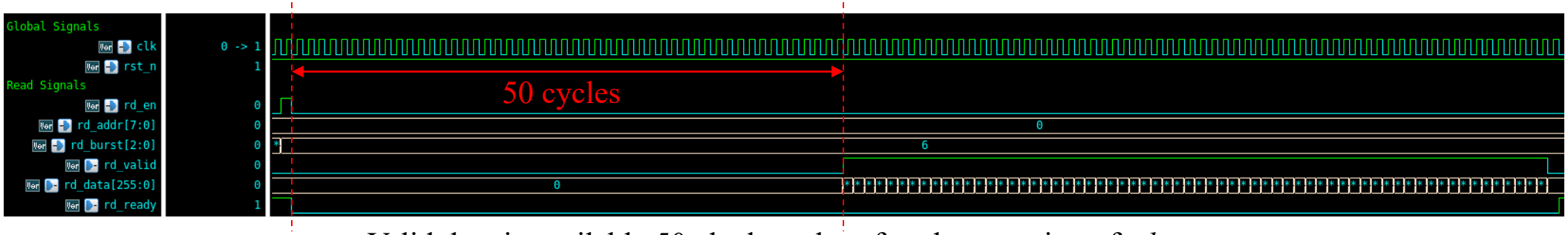
Signal	I/O	Bit Width	Definition
Global Signals			
clk	input	1	Clock signal. Pseudo RAM is **negative edge triggered**.
rst_n	input	1	Active-low asynchronous reset.
Read Signals			
rd_en	input	1	Read enable. Assert high to initiates a read transaction.
rd_addr	input	8	Read address. The starting address for the read operation.
rd_burst	input	3	Burst length. Read 2^{rd_burst} for each read operation.
rd_valid	output	1	Read data valid. High when *rd_data* is valid.
rd_data	output	256	Read data.
rd_ready	output	1	High when RAM is prepared to accept a new read request.

Pseudo RAM : System Specifications

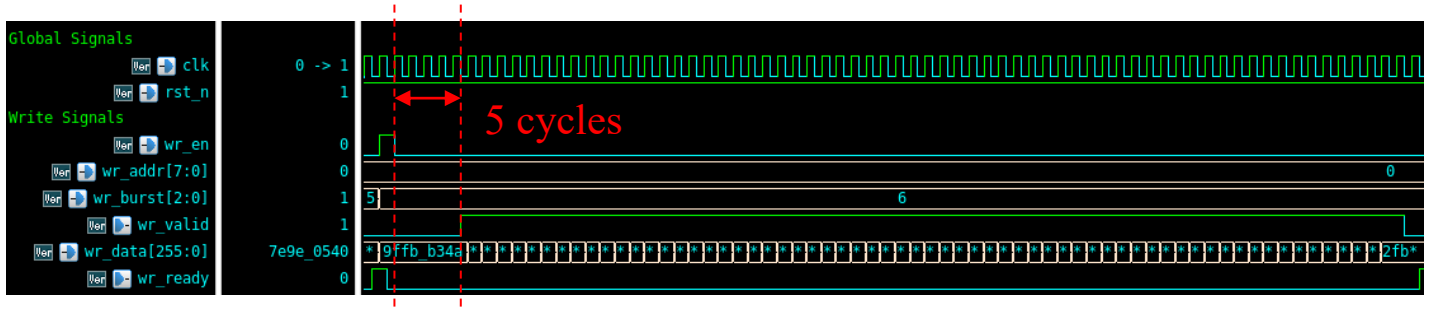
Signal	I/O	Bit Width	Definition
Write Signals			
wr_en	input	1	Write enable. Assert high to initiates a write transaction.
wr_addr	input	8	Write address. The starting address for the write operation.
wr_burst	input	3	Burst length. Write 2^{rd_burst} for each write operation.
wr_valid	output	1	Write data valid. High when *wr_data* is valid.
wr_data	input	256	Write data.
wr_ready	output	1	High when RAM is prepared to accept a new write request.

Pseudo RAM : System Specifications

- Timing & Latency Profile:
  - **Read Latency**: 50 cycles.
  - **Write Latency**: 5 cycles.



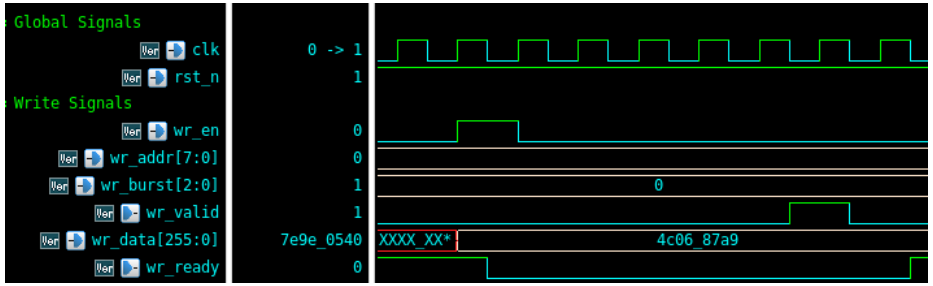
Valid data is available 50 clock cycles after the assertion of *rd_en*



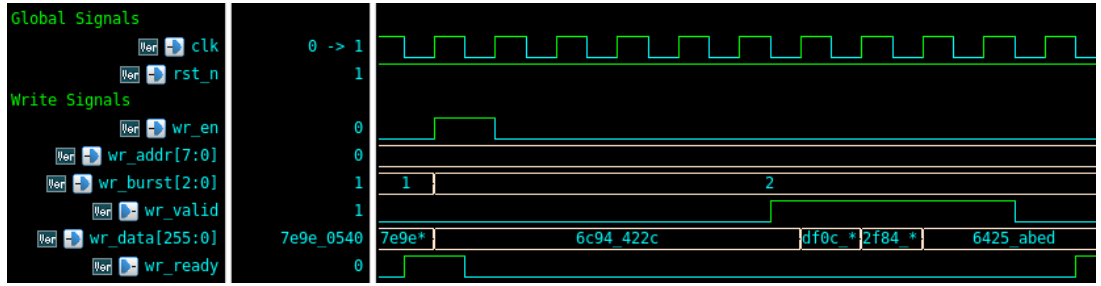
Data is committed to the RAM 5 clock cycles after the assertion of *wr_en*

Pseudo RAM : System Specifications

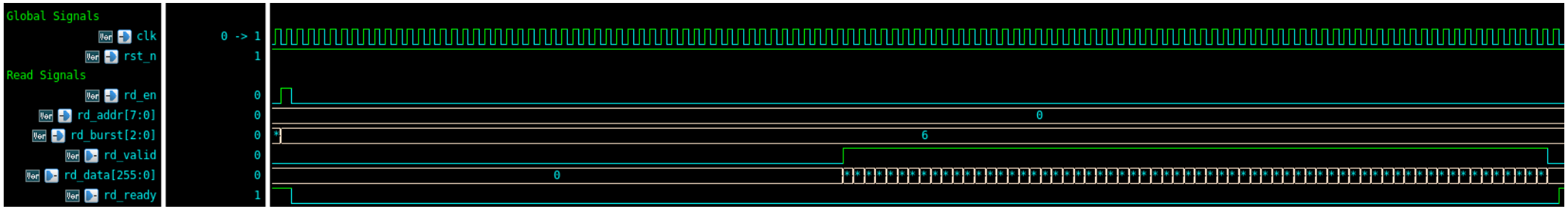
- Dynamic Burst Mode:
  - **Configurable Throughput**: Supports variable burst lengths in both Read and Write to optimize data transfer efficiency.
  - **Burst Range**: 2^0 to 2^7 (1 to 128 words per Read).



Single Write (*wr_burst*=0, write 2^0 data)

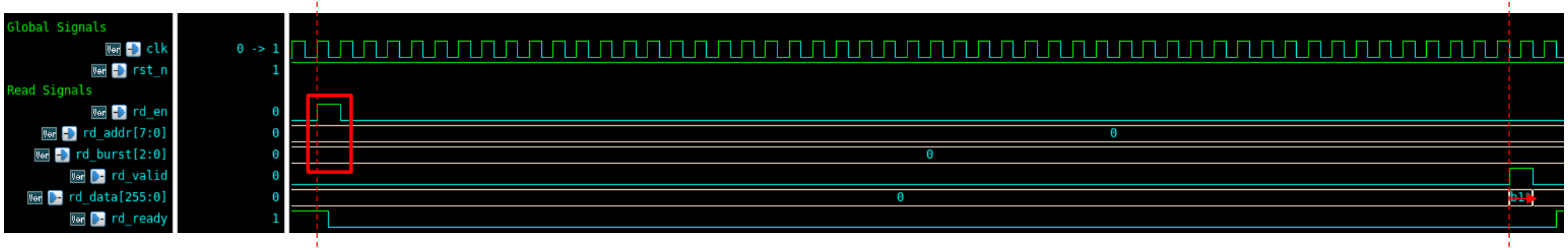


Burst Write (*wr_burst*=2, write 2^2 data)



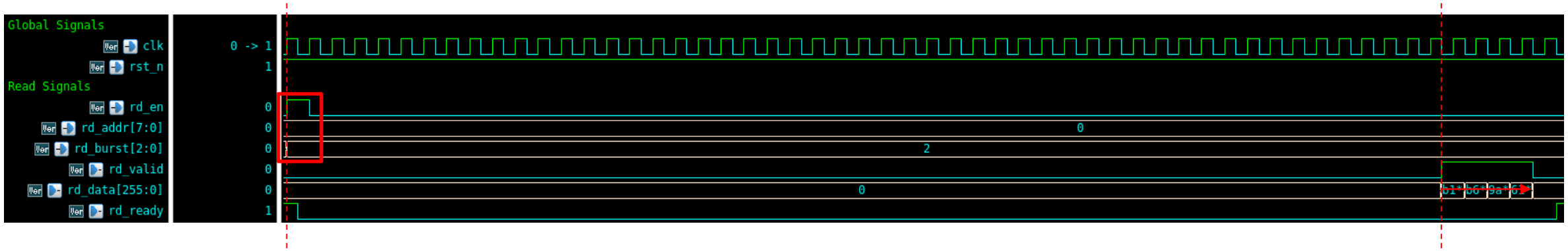
Burst Read (*rd_burst*=6, write 2^6 data)

Example Usage : Single Read



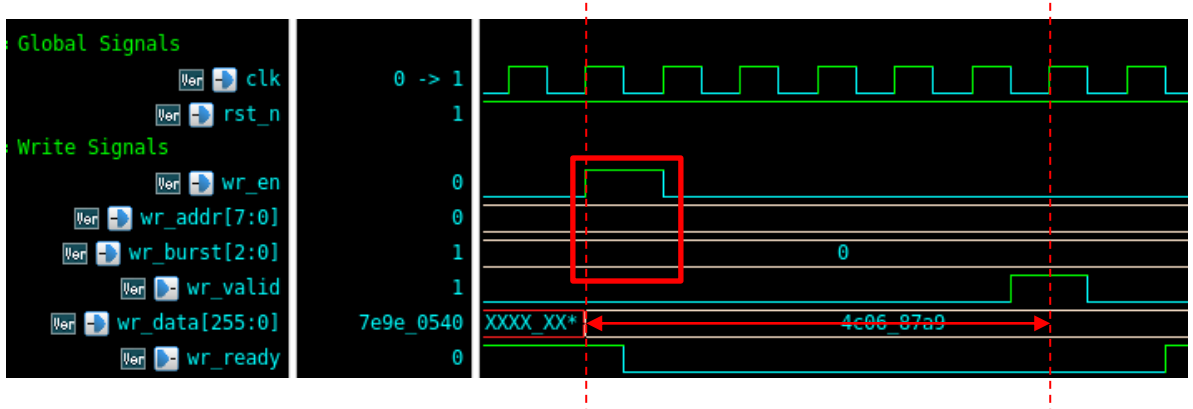
- Step 1: When *rd_ready* = 1, pull *rd_en* high and give *rd_addr*, *rd_burst*(= 0) at the same time to initiate a Read.
- Step 2: Valid read data arrives when *rd_valid* = 1.

Example Usage : Burst Read



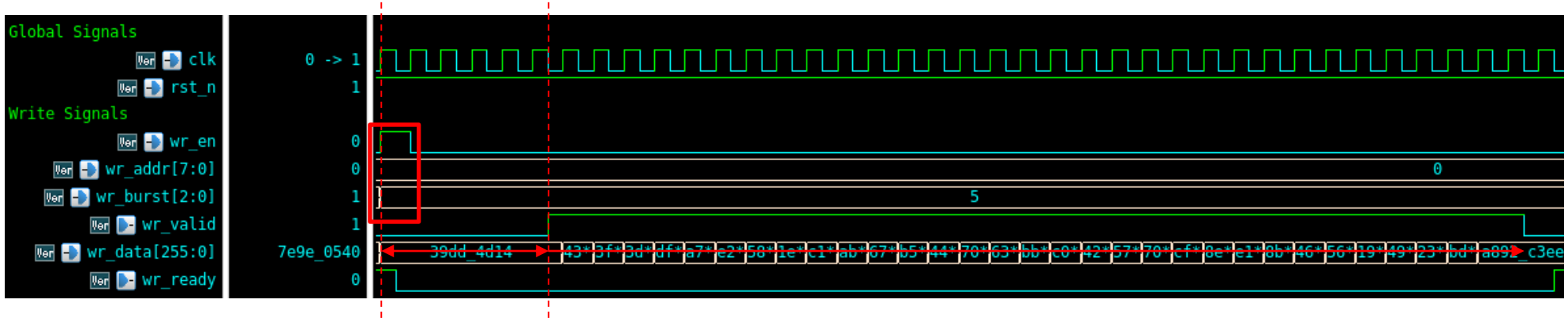
- Step 1: When *rd_ready* = 1, pull *rd_en* high and give the start address *rd_addr*, expected burst length *rd_burst*(> 0) at the same time to initiate a burst Read.
- Step 2: Valid read data arrive when *rd_valid* = 1.

Example Usage : Single Write



- Step 1: When *wr_ready* = 1, pull *wr_en* high and give *wr_addr*, *wr_burst*(= 0) at the same time to initiate a Write.
- Step 2: Hold *wr_data* until *wr_valid* = 1 (the data will be written at the posedge of *wr_valid*).

Example Usage : Burst Write



- Step 1: When *wr_ready* = 1, pull *wr_en* high and give the start address *wr_addr*, expected burst length *wr_burst*(> 0) at the same time to initiate a Write.
- Step 2: Hold *wr_data* until *wr_valid* = 1, and then switch to next write data every cycle.